System Design Document — Agentic AI System

Assignment: Agentic AI System for Multi-Step Tasks

1. Project Overview

This document describes the system design of an Agentic AI System capable of handling complex multi-step tasks. The system decomposes user tasks, routes them through specialised AI agents via an async message queue, and streams partial responses back to the user in real time.

1.1 Problem Statement

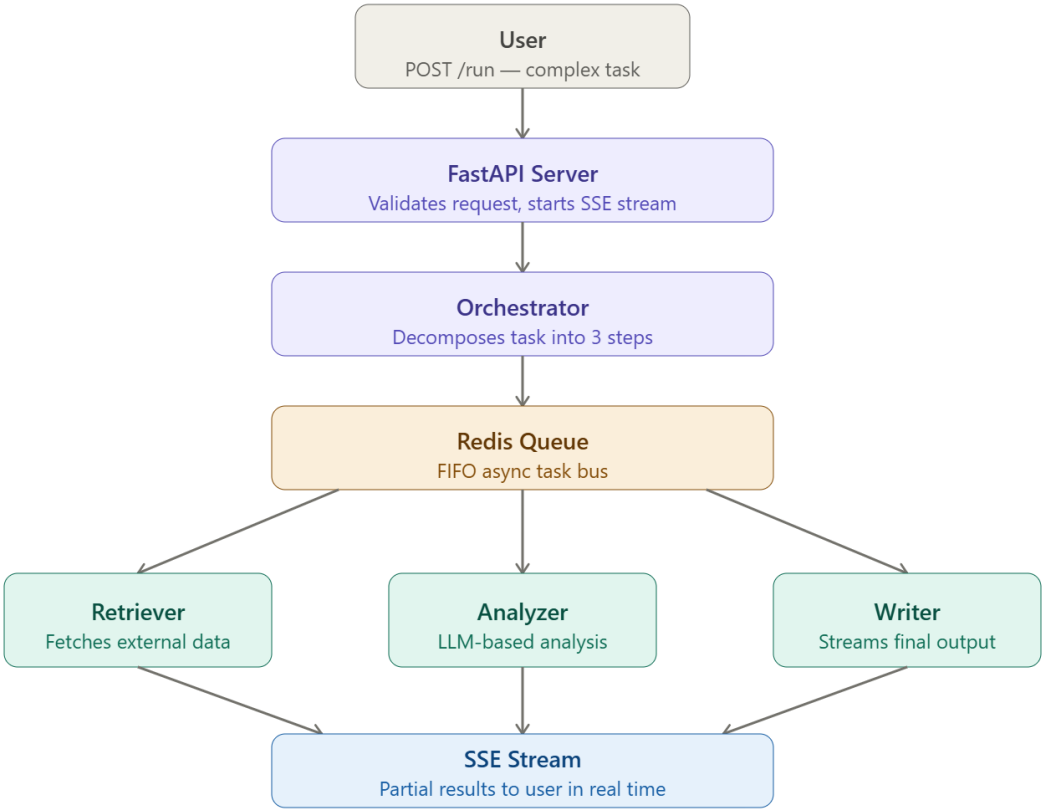
Traditional AI systems process user queries in a single pass. Complex tasks — such as researching a topic, analysing findings, and producing a structured report — require multiple coordinated steps. This system solves that by orchestrating a pipeline of specialised agents, each responsible for one part of the task.

1.2 Goals

- Accept complex multi-step user tasks via REST API
- Decompose tasks into discrete steps automatically
- Execute each step using a specialised agent (Retriever, Analyser, Writer)
- Communicate between components using an async message queue (Redis)
- Stream partial responses to the user via Server-Sent Events (SSE)
- Handle failures gracefully with retry logic and dead-letter queuing

2. System Architecture

2.1 High-Level Architecture Flow



2.2 Component Overview Table

Component	Role	Technology
FastAPI Server	Accepts user tasks, streams responses via SSE	FastAPI + Uvicorn
Orchestrator	Decomposes tasks, dispatches to agents, aggregates results	Python asyncio
Redis Queue	Async FIFO message bus between orchestrator and agents	Redis LPUSH/BRPOP
Retriever Agent	Fetches external data, documents, and web content	httpx + async
Analyzer Agent	Processes and summarises retrieved content via LLM	OpenAI API

Writer Agent	Generates final structured output with streaming	OpenAI API + SSE
Dead-Letter Queue	Stores failed tasks after max retries	Redis List

3. Component Design

3.1 FastAPI Server (server.py)

The server is the entry point for all client interactions. It exposes three endpoints and manages the application lifecycle, including Redis connection setup and teardown.

Endpoint	Method	Description
/health	GET	Returns system health status
/run	POST	Accepts task, returns SSE stream of agent progress
/queue/status	GET	Returns current queue depths for monitoring

3.2 Orchestrator (orchestrator.py)

The Orchestrator is the brain of the system. It receives the user task, breaks it into steps, dispatches each step to the appropriate agent, and aggregates results into the final streaming response.

Task Decomposition

Every task is decomposed into exactly 3 sequential steps:

- Step 1: RETRIEVE: Fetch relevant data from external sources
- Step 2: ANALYZE: Process and extract key insights from retrieved data
- Step 3: WRITE: Generate a structured, formatted final response

Context Passing

Each step's output is stored in a shared context dictionary and passed to the next step. The Retriever's output becomes the Analyser's input; the Analyser's output becomes the Writer's input.

3.3 Base Agent (base_agent.py)

Parameter	Value	Purpose
max_retries	3	Maximum attempts before marking the task failed
retry_delay	1.0s	Base delay — doubles each retry (1s, 2s, 4s)
dead_letter	True flag	Failed tasks flagged for the dead-letter queue

3.4 Specialised Agents

Retriever Agent

- Responsible for fetching external data based on the user query
- Supports mock mode (simulated data) and real HTTP mode (httpx)
- Returns structured data including retrieved_data, source, and query fields

Analyzer Agent

- Uses OpenAI GPT-4o-mini to analyse and summarise retrieved content
- Falls back to mock analysis if no API key is configured
- Returns analysis text and token usage statistics

Writer Agent

- Generates final formatted response using OpenAI GPT-4o-mini
- Supports both batch execution (execute) and streaming (stream_execute)
- In streaming mode, yields output word by word for real-time display

3.5 Redis Message Queue (redis_queue.py)

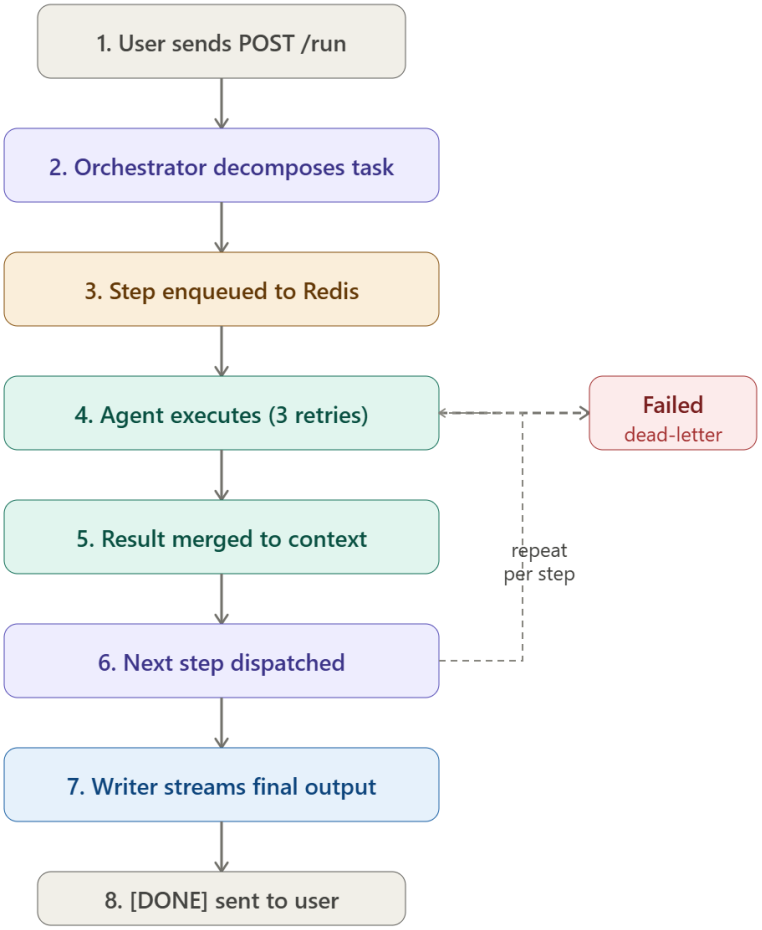
The queue acts as the communication backbone between the Orchestrator and agents. It uses Redis Lists with LPUSH (enqueue) and BRPOP (dequeue) for FIFO ordering.

Queue Name	Purpose
task_queue	Main queue for dispatching steps to agents
dead_letter_queue	Stores failed tasks after all retries were exhausted.

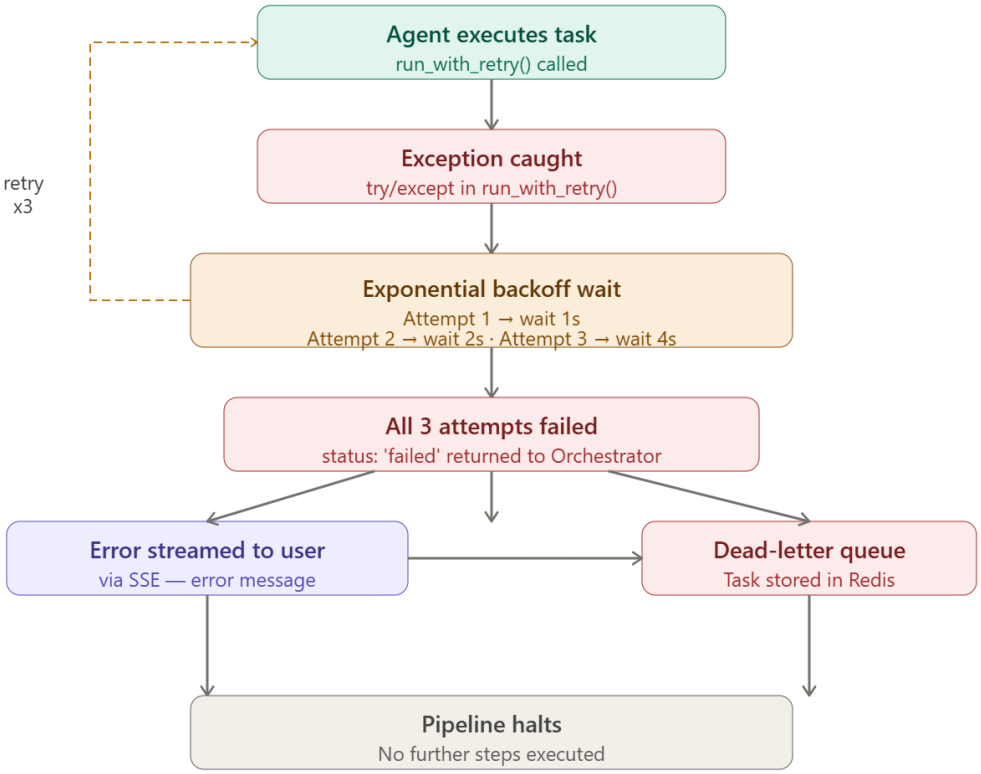
Mock Mode: If Redis is unavailable, the queue runs in mock mode — all operations are logged locally, and the system continues to function without a real Redis instance.

4. Data Flow

4.1 Request Lifecycle



4.2 Failure Handling Flow



5. Technology Stack

Technology	Version	Justification
FastAPI	0.136.0	Async-native, auto API docs, supports SSE
Uvicorn	0.44.0	ASGI server for async FastAPI
Redis	7.4.0	Sub-ms FIFO queue with LPUSH/BRPOP
OpenAI SDK	2.32.0	Async LLM inference with streaming
httpx	0.28.1	Async HTTP client for agent-to-API calls
SSE-Starlette	3.3.4	Server-Sent Events for FastAPI
Pydantic	2.13.2	Request/response validation
asyncio	built-in	Native async concurrency — no extra deps

6. Scalability Considerations

6.1 Current Limitations

- Single shared Redis queue — all agent types compete for the same queue
- One Orchestrator per request — no distributed state management
- Sequential pipeline — steps cannot run in parallel, even if independent
- No horizontal scaling of agent workers out of the box

6.2 Scaling Strategy

Bottleneck	Solution
Shared queue saturation	Per-agent queues: retriever_queue, analyzer_queue, writer_queue
Single orchestrator	Distributed scheduler: Celery, Temporal, or Prefect
Sequential steps	DAG task planner for parallel independent sub-tasks
Single Redis	Redis Cluster or Redis Streams with consumer groups
Agent worker capacity	Kubernetes HPA — scale agent pods per queue depth

7. API Reference

GET /health

Returns the health status of the system.

Response: { "status": "ok", "service": "agentic-ai-system" }

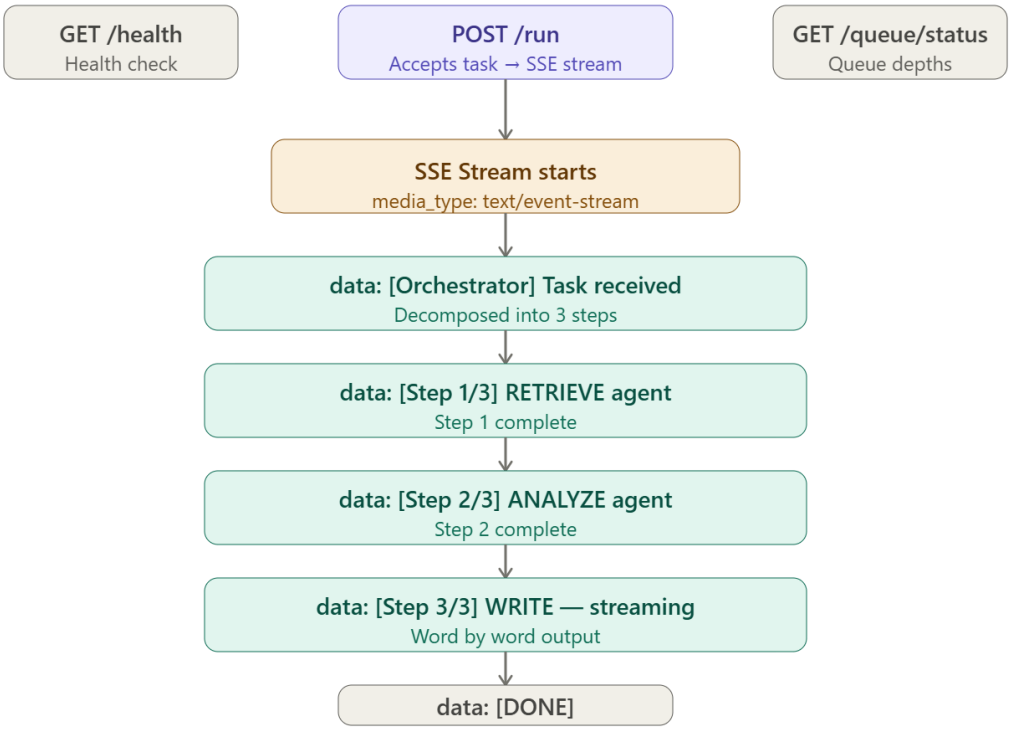
POST /run

Accepts a complex user task and returns a streaming SSE response showing agent progress and final output.

Request Body: { "task": "string — the user's complex task" }

Response: text/event-stream — SSE chunks including step progress and final streamed text

Example SSE output:



GET /queue/status

Returns the current depth of task queue and dead-letter queue for monitoring.

Response: { "task_queue": 0, "dead_letter_queue": 0 }

8. Deployment

8.1 Local Development

- Python 3.11+ with virtual environment
- Redis via Docker: docker run -d --name redis -p 6379:6379 redis:7-alpine
- Server: uvicorn src.server:app --reload --port 8000
- Mock mode available without Redis or OpenAI API key

8.2 Docker Compose

Full containerized deployment with docker-compose up --build. Includes Redis service with healthcheck and API service with environment variable injection.

8.3 Environment Variables

Variable	Default	Description
OPENAI_API_KEY	(empty)	System works in mock mode without it
REDIS_HOST	localhost	Redis server hostname
REDIS_PORT	6379	Redis server port